

MEMORIA PRÁCTICA 1

Elena Abreu Fernández y Tsan-Yu Wu

● Opción -d

Lo primero que se pide en esta práctica es realizar un método tal que al proporcionar “-d” como parámetro de entrada devuelva una tabla con el nombre, tipo, codificación e idioma del fichero que se analiza.

```
private static void optionD( File file ) throws Exception{

    Tika tika = new Tika();
    Metadata metadata = new Metadata();
    InputStream is = new FileInputStream(file);
    ParseContext parsecontext = new ParseContext();
    AutoDetectParser parser = new AutoDetectParser();
    BodyContentHandler handler = new BodyContentHandler(-1);

    //con autodetectParser obtenemos los metadatos y contenido del doc
    parser.parse(is,handler,metadata,parsecontext);

    //obtenemos nombre
    String name = file.getName();

    //obtenemos tipo
    String type = tika.detect(file);

    //obtenemos codificación
    String encoding = metadata.get("Content-Encoding");

    //obtenemos el idioma
    LanguageDetector detector = new OptimaizeLangDetector().loadModels();
    LanguageResult result = detector.detect(handler.toString());
    String language = result.getLanguage();

    //imprimimos en forma de tabla
    System.out.println(name + "\t" + type + "\t" + encoding + "\t" + language);

}
```

Vemos que para obtener el nombre basta trabajar sobre el fichero, mientras que el tipo nos proporciona un objeto Tika, la codificación la obtenemos a partir de los metadatos del archivo, y el lenguaje lo extraemos usando distintos métodos de las bibliotecas Tika. Un ejemplo de salida de este método sería el siguiente :

fantasma.txt text/plain UTF-8 fr

● Opción -l

A continuación, buscamos crear un método que nos devuelva todos los enlaces que se encuentran en un documento cuando pasamos “-l” como parámetro de entrada.

```
private static void optionL(File file) throws Exception{
    LinkContentHandler linkhandler = new LinkContentHandler();
    ContentHandler texthandler = new BodyContentHandler(-1);
    TeeContentHandler teehandler = new TeeContentHandler(linkhandler, texthandler);
    FileInputStream is = new FileInputStream(file);

    Metadata metadata = new Metadata();
    ParseContext parseContext = new ParseContext();
    AutoDetectParser parser = new AutoDetectParser();

    parser.parse(is, teehandler, metadata, parseContext);

    System.out.println("links: \n" + linkhandler.getLinks());
}
```

Para ello creamos un linkhandler que detectará los enlaces durante el parseo, un texthandler que extraerá el texto, y teehandler que agrupa ambos. Con el parser de tipo AutoDetectParser, que detecta automáticamente el tipo de archivo, analizaremos el documento para finalmente extraer los links correspondientes con getLinks(). Un ejemplo de salida de este método es :

links:

[, , Freeditorial.com]

● Opción -t

Al final, generamos un fichero CSV con las palabras y sus frecuencias en orden decreciente de frecuencia, si el fichero de Yerma es .txt (texto plano)

```
private static void optionT(File file) throws Exception {

    // Creamos una instancia de Tika con la configuracion por defecto
    Tika tika = new Tika();

    File f = new File(file);

    // Obtenemos el tipo de fichero
    String type = tika.detect(f);

    // Chequeamos si el fichero es de texto plano
    if (type == "text/plain") {
        // Creamos un HashMap para almacenar las palabras y sus frecuencias (sin orden)
        HashMap<String, Integer> hm = new HashMap<String, Integer>();
        InputStream is = new FileInputStream(f);
        String text = tika.parseToString(is);

        // Seperamos el texto en palabras, contamos sus frecuencias y almacenamos en el HashMap
        for (String word: text.split("\\W+")){
            word = word.toLowerCase();
            if (hm.containsKey(word)) {
                hm.put(word, hm.get(word) + 1);
            } else {
                hm.put(word, 1);
            }
        }

        // Preparamos el HashMap ordenado por frecuencia
        Map<String, Integer> hm1 = sortByValue(hm);
    }
}
```

Aquí usamos el tipo “HashMap” para realizar un diccionario en el que almacenamos la palabra y su frecuencia. Después de contar, ordenamos las frecuencias en orden decreciente por una subfunción:

```
// Metodo para ordenar el HashMap por valor
static HashMap<String, Integer> sortByValue(HashMap<String, Integer> hm) {
    List<Map.Entry<String, Integer> > list =
        new LinkedList<Map.Entry<String,Integer> >(hm.entrySet());

    // Ordenamos decreciente por valor
    Collections.sort(list, new Comparator<Map.Entry<String, Integer> >() {
        public int compare(Map.Entry<String,Integer> o1, Map.Entry<String, Integer> o2) {
            return (o2.getValue()).compareTo(o1.getValue());
        }
    });

    // Recreamos un HashMap con los elementos ordenados
    HashMap<String, Integer> temp = new LinkedHashMap<String, Integer>();
    for (Map.Entry<String, Integer> aa: list) {
        temp.put(aa.getKey(), aa.getValue());
    }
    return temp;
}
```

Cuando tengamos el HashMap ordenado, crearemos un fichero para escribir el HashMap conseguido, si no ya existe uno. Escribimos luego “Text;Size” en la primera fila y lo que tenemos en el HashMap en el resto

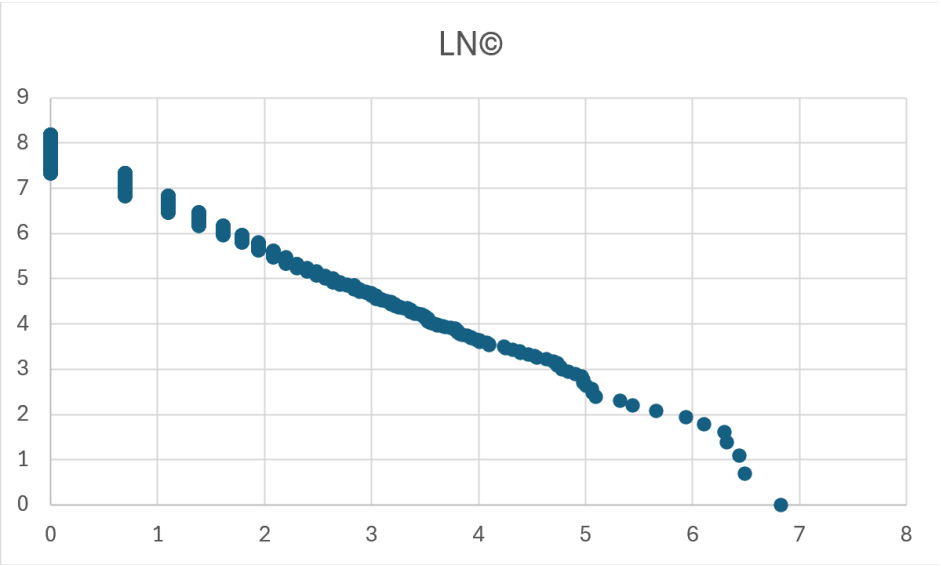
```
String filename = "frecuencia.csv";
// Creamos el fichero csv, chequeando si ya existe
try {
    File output = new File(filename);
    if (output.createNewFile()) {
        System.out.println("File created: " + output.getName());
    } else {
        System.out.println("File already exists.");
    }
} catch (IOException e) {
    System.out.println("An error occurred.");
    e.printStackTrace(); // Print error details
}

// Luego, escribimos el HashMap ordenado en el fichero creado
try {
    FileWriter writer = new FileWriter(filename);
    // Escribimos la primera fila en el fichero
    writer.write("Text;Size\n");
    for (String word: hm1.keySet()) {
        // Escribimos cada palabra y su frecuencia en el fichero
        writer.write(word + ";" + hm1.get(word) + "\n");
    }
    writer.close();
    System.out.println("Successssfully wrote to the file.");
} catch (IOException e) {
    System.out.println("Error occurred while writng.");
    e.printStackTrace();
}
} else {
    System.out.println("El fichero " + file + " no es de texto plano");
}
}
```

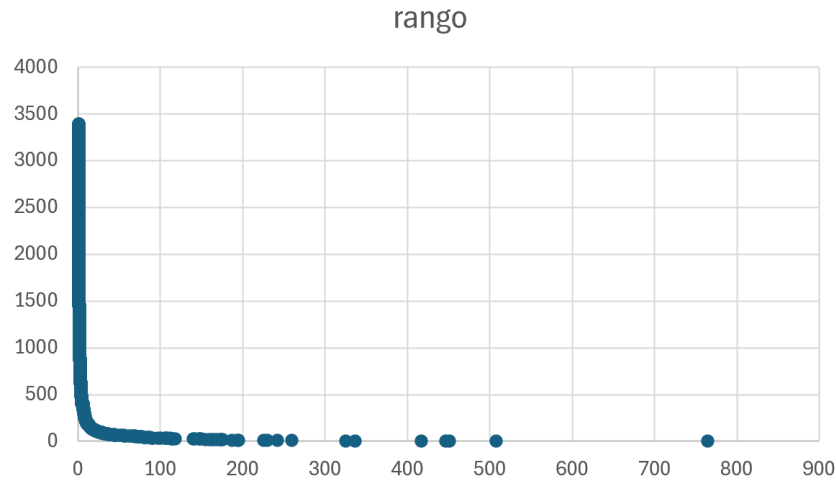
Yerma_poema_trágico_en_3_actos_dividido_en_2_cuadros_cada_uno_en_prosa_y_vers
o.txt

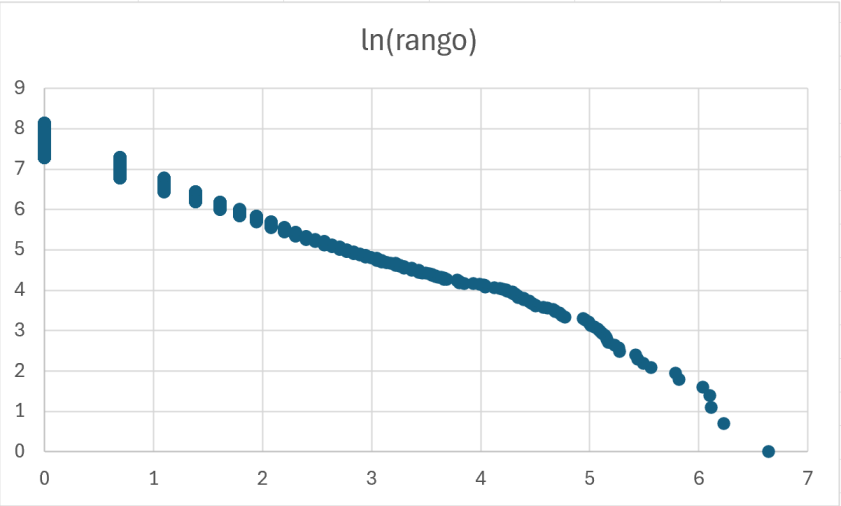


The scatter plot, titled "Rango", displays the distribution of values. The vertical axis, labeled "Área del gráfico", ranges from 0 to 4000. The horizontal axis ranges from 0 to 1000. The data points are concentrated at the origin (0,0), with a few scattered points at higher "Rango" values, indicating a highly skewed distribution.

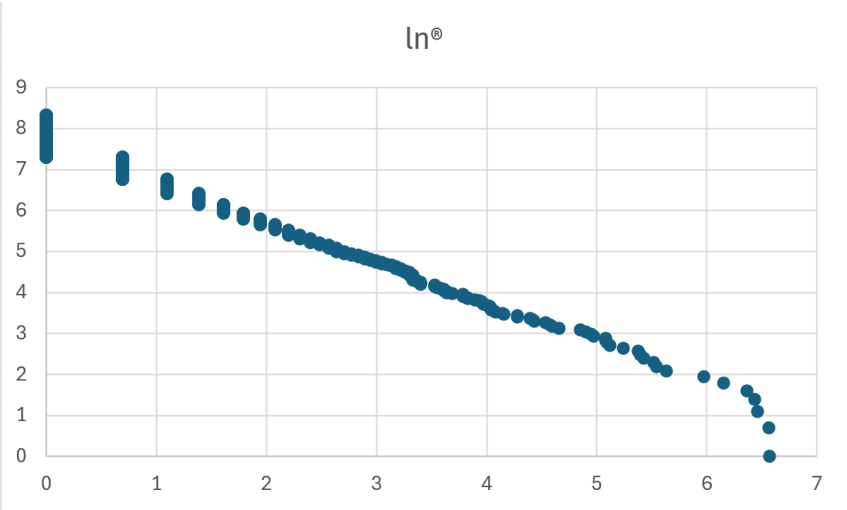
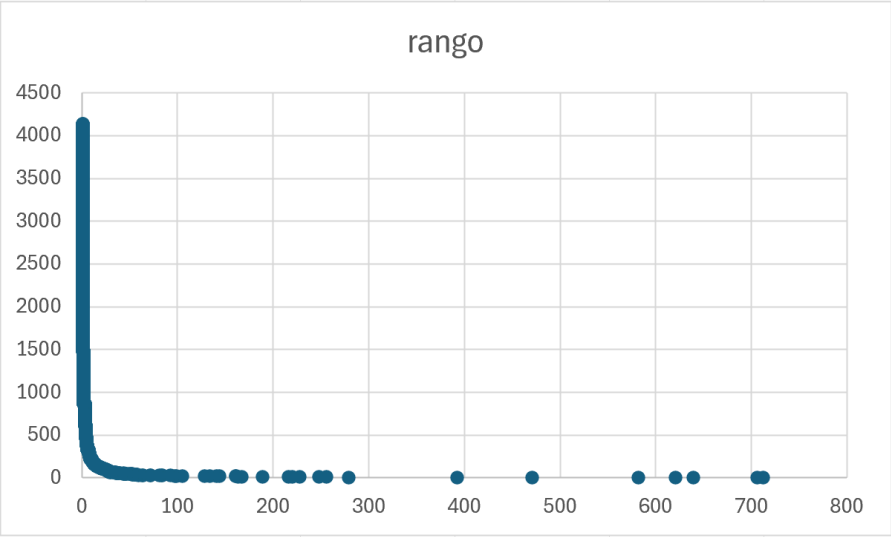


Gráficos de “El fantasma de la ópera” en francés :





Gráficos de “La divina comedia” en español :



● Trabajo en grupo

Elena : código para la opción -d y la opción -l, gráficos de la ley de Zipf, partes correspondientes de la memoria.

Tsan-Yu : código para la opción -t y nube de palabras, partes correspondientes de la memoria.